

REST API NOTES

CHAPTER 1

Page 1/10

1. REST API FUNDAMENTALS

★ What is an API?

API (Application Programming Interface) is a set of rules and protocols that allows two software applications to communicate with each other.

★ What is REST API?

REST (Representational State Transfer) is an architectural style for designing networked applications. RESTful APIs use **HTTP** methods to perform operations on resources and return data in standard formats like **JSON** or **XML**.

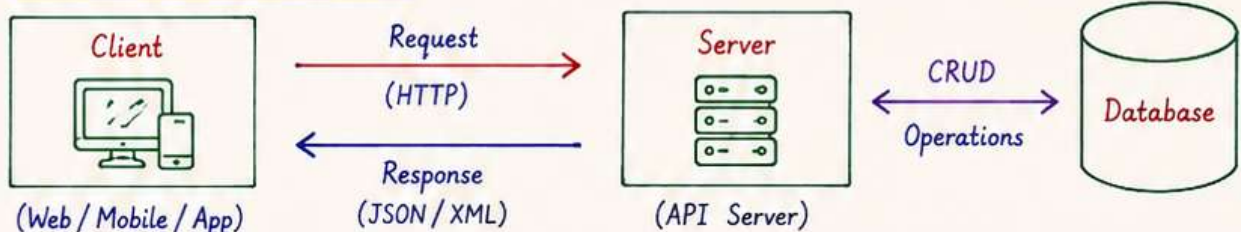
★ Real World Examples

- Swiggy - fetch restaurants, place order
- Instagram - get posts, like, comment
- UPI Apps - check balance, transfer money
- Weather Apps - get weather details
- E-commerce - manage products, orders

★ REST vs SOAP (Quick Comparison)

REST	SOAP
Architectural Style	Protocol
Uses HTTP	Uses XML over various protocols
Lightweight	Heavyweight
JSON, XML, etc.	Only XML
Stateless	Can be Stateful
Easy to use & scalable	Complex & less flexible
Widely used in Web & Mobile Apps	Used in Enterprise Applications

★ Client - Server Architecture



★ Request → Response Flow



REST API NOTES

CHAPTER 1

Page 2/10

@aman_views

2. HTTP METHODS

★ What are HTTP Methods?

HTTP methods (also called verbs) are used to perform operations on a resource identified by a URL.

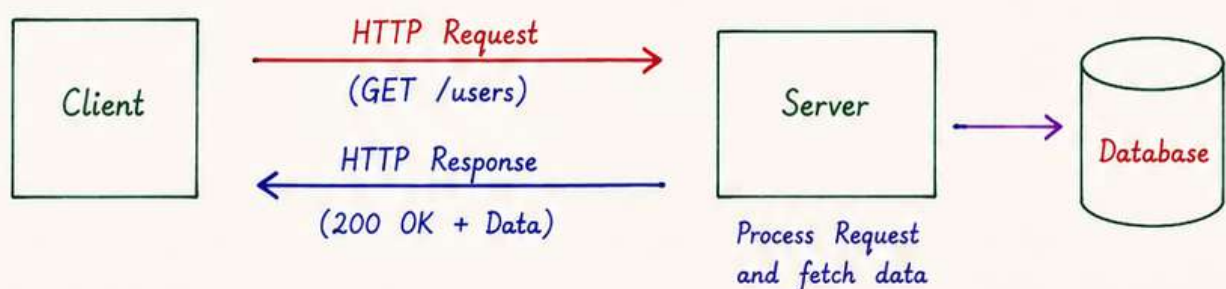
★ Common HTTP Methods

1. **GET** - Retrieve data from server
2. **POST** - Create a new resource
3. **PUT** - Update or replace a resource
4. **PATCH** - Partially update a resource
5. **DELETE** - Remove a resource
6. **HEAD** - Get headers of a resource
7. **OPTIONS** - Get supported methods

★ HTTP Methods Comparison

Method	Purpose	Idempotent?	Safe?	Request Body	Example
GET	Retrieve data	Yes	Yes	No	GET /users
POST	Create new data	No	No	Yes	POST /users
PUT	Update/Replace entire resource	Yes	No	Yes	PUT /users/1
PATCH	Update part of a resource	No	No	Yes	PATCH /users/1
DELETE	Delete a resource	Yes	No	No	DELETE /users/1
HEAD	Get response headers	Yes	Yes	No	HEAD /users
OPTIONS	Get supported methods	Yes	Yes	No	OPTIONS /users

★ Request Flow (Example)



REST API NOTES

CHAPTER 1

Page 3/10

@aman_views

3. HTTP REQUEST STRUCTURE

★ What is an HTTP Request?

An HTTP request is a message sent by the client to the server to perform an operation on a **resource**.

★ Components of HTTP Request

1. **Method** - The HTTP method (GET, POST, etc.)
2. **URL** - The endpoint of the resource
3. **Headers** - Additional information about the request
4. **Query Parameters** - Optional data sent in the URL
5. **Path Parameters** - Values included in the URL path
6. **Body** - Data sent to the server (for POST, PUT, PATCH)

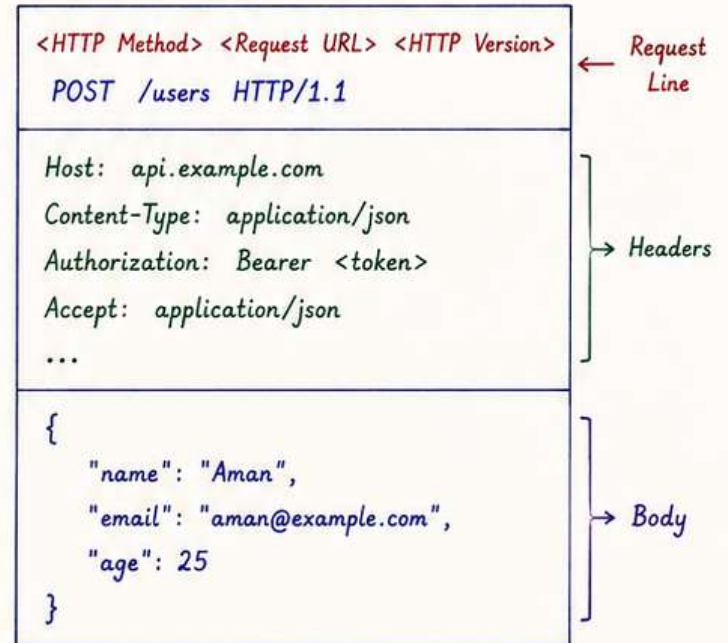
★ Types of Parameters

- **Query Parameter** - Sent after ? in URL
Example: `/users?page=1&size=10`
- **Path Parameter** - Sent as part of the URL path
Example: `/users/10`
- **Request Body** - Sent in the body of request
Example: `{ "name": "Aman" }`

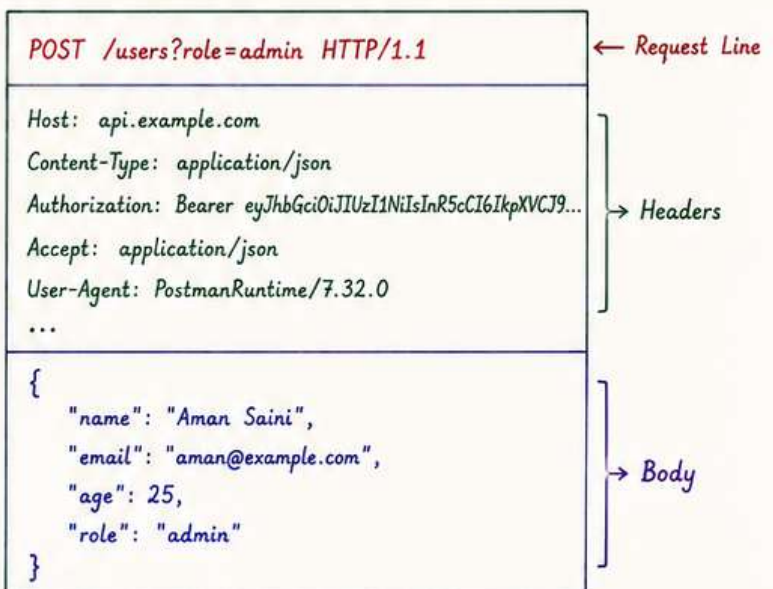
★ Key Points

- GET, HEAD, OPTIONS do not have a body.
- POST, PUT, PATCH can have a body.
- Headers provide metadata about the request.
- Query parameters are optional.
- Path parameters are part of the resource path.

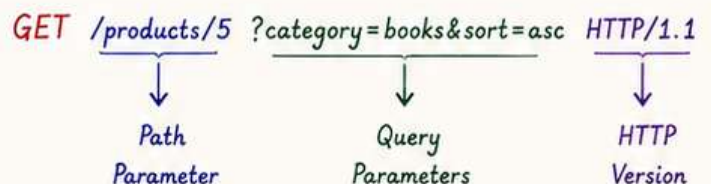
★ Request Structure Overview



★ Complete HTTP Request Example



★ Simple Example



REST API NOTES

CHAPTER 1

Page 4/10

4. HTTP RESPONSE & STATUS CODES

@aman_views

★ What is an HTTP Response?

An HTTP response is the message sent by the server back to the client in response to a request.

★ Response Structure

An HTTP response has three main parts:

1. **Status Line** - Contains HTTP version, status code and reason phrase.
2. **Headers** - Provides metadata about the response.
3. **Body** - Contains the actual data returned by the server.

Structure Diagram

Status Line (HTTP/1.1 200 OK)

Headers (Content-Type, Date, etc.)

Body
(JSON / XML / Plain Text / etc.)

★ Response Example

HTTP/1.1 200 OK

Content-Type: application/json
Date: Wed, 22 May 2024 10:30:00 GMT
Server: Apache/2.4.58
Content-Length: 72

```
{
  "id": 1,
  "name": "Aman Saini",
  "email": "aman@example.com",
  "age": 25
}
```

★ Common HTTP Status Codes

Status codes are returned by the server to indicate the result of the request.

Code	Category	Meaning	When it Occurs
200	Success	OK	Request is successful
201	Success	Created	Resource created successfully
204	Success	No Content	Request successful, no data to return
400	Client Error	Bad Request	Invalid request (syntax, data, etc.)
401	Client Error	Unauthorized	Authentication required or failed
403	Client Error	Forbidden	Authenticated but no access
404	Client Error	Not Found	Resource not found on server
409	Client Error	Conflict	Conflict with current state of resource
500	Server Error	Internal Server Error	Something went wrong on server

★ Request → Response Flow



★ Key Points

- Status codes help understand the result of request.
- 2xx → Success
- 4xx → Client side errors
- 5xx → Server side errors
- Response body format is usually JSON.

REST API NOTES

CHAPTER 1

Page 5/10

@aman_views

5. AUTHENTICATION & AUTHORIZATION

★ Authentication vs Authorization

- **Authentication** - Verifies who you are. (Identity Verification)
- **Authorization** - Determines what you can access. (Permission Verification)

★ Common Authentication Methods

1. Basic Authentication

Uses username and password encoded in Base64.

2. Bearer Token (Token Based)

Client sends a token in Authorization header.

3. JWT (JSON Web Token)

Token with payload, header and signature. Stateless and widely used.

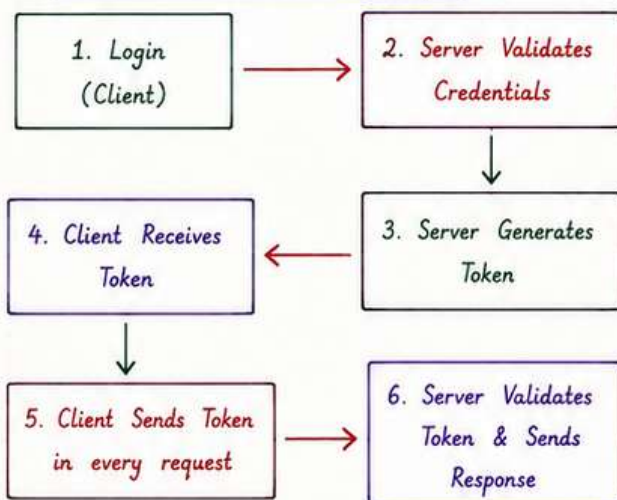
4. OAuth 2.0

Authorization framework that allows third-party access without sharing credentials.

5. API Key

A unique key is sent with each request to identify the client.

★ Authentication Flow (Example)



★ How Token Based Authentication Works

1. User logs in with credentials.
2. Server validates and generates a token.
3. Client stores the token.
4. Client sends the token in Authorization header for every request.
5. Server validates the token and processes the request.

★ Example: Bearer Token in Header

GET /products HTTP/1.1

Host: api.example.com

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Content-Type: application/json

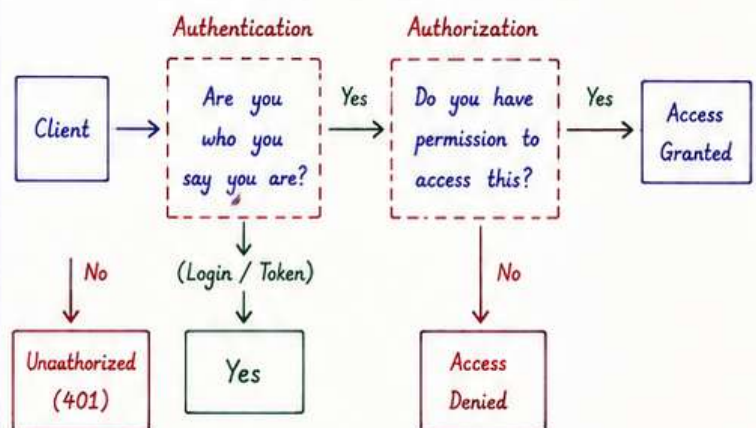
Accept: application/json

...

★ Key Points

- Authentication happens first.
- Authorization happens after authentication.
- Never store plaintext passwords.
- Use HTTPS to secure data in transit.
- Tokens should have expiry time.
- Validate token on every request.

★ Authentication vs Authorization (Flow)



REST API NOTES

CHAPTER 1

Page 6/10

@aman_views

6. REST API BEST PRACTICES

★ What is REST Best Practices?

Following best practices makes APIs more consistent, scalable, secure and easy to use.

★ 1. Resource Naming

- Use nouns to represent resources.
- Use plural for collection resources.

Examples:

/users , /orders , /products

★ 2. URI Design

- Use clear and meaningful URLs.
- Use / to represent hierarchy.
- Avoid verbs in URL.

Examples:

/users

/users/10

/users/10/orders

★ 3. HTTP Methods Best Practices

- **GET** - Retrieve data (Safe & Idempotent)
- **POST** - Create new resource (Not Idempotent)
- **PUT** - Replace entire resource (Idempotent)
- **PATCH** - Update part of resource
- **DELETE** - Remove resource (Idempotent)

★ 4. Stateless API

Each request from client must contain all information needed to process the request.

★ 5. API Versioning

Use versioning in URI to manage changes.

Examples:

/api/v1/users

/api/v2/users

★ Good vs Bad Examples

Good Example

- ✓ GET /users
- ✓ GET /users/10
- ✓ POST /users
- ✓ PUT /users/10
- ✓ DELETE /users/10

vs

Bad Example

- ✗ GET /getUsers
- ✗ POST /createUser
- ✗ GET /users/get/10
- ✗ POST /updateUser
- ✗ GET /deleteUser/10

★ 6. Use Proper Status Codes

Always return appropriate HTTP status codes to indicate the result of request.

★ 7. Data Format

Use JSON for request and response body. It is lightweight and easy to read.

```
{
  "id": 1,
  "name": "Aman",
  "email": "aman@example.com"
}
```

★ 8. Idempotent Methods

Multiple requests should have same effect as a single request.

Idempotent: GET, PUT, DELETE

Non-Idempotent: POST, PATCH

★ 9. Pagination

Use query parameters for pagination.

Example: /users?page=1&size=10

★ 10. Filtering, Sorting & Searching

Use query parameters for filtering and sorting.

Example:

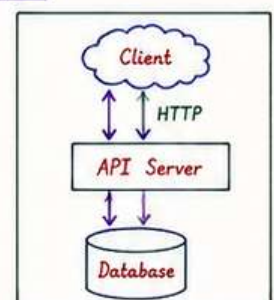
/products?category=books&sort=asc&search=java

★ Best Practices Summary

Practice	Why it matters
Use nouns in URL	Makes API easy to understand
Use proper HTTP methods	Represents correct operations
Use status codes	Improves error handling
Use versioning	Handle API changes smoothly
Keep API stateless	Improve scalability
Use JSON	Lightweight & widely supported

★ RESTful API Design Principles

- Client - Server Architecture
- Stateless
- Cacheable (if possible)
- Layered System
- Uniform Interface
- Code on Demand (optional)



REST API NOTES

CHAPTER 1

Page 7/10

7. BUILDING REST APIs (SPRING BOOT) - PART 1

★ Overview

Spring Boot makes it very easy to build **RESTful** APIs using annotations.

The main entry point is the **Controller** class.

★ Important Annotations

1. @RestController

- Marks the class as a REST controller.
- Combination of **@Controller** + **@ResponseBody**.

2. @RequestMapping

- Maps HTTP requests to handler methods.
- Can be used at class level or method level.

3. @GetMapping

- Handles HTTP GET requests.
- Used to fetch data.

4. @PostMapping

- Handles HTTP POST requests.
- Used to create new resource.

★ Basic Structure

```
@RestController
@RequestMapping("/api")
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello, REST API!";
    }
}
```

★ What Happens?

1. Client sends GET request to /api/hello
2. Spring Boot matches the request to hello() method
3. Method executes and returns the response
4. Spring Boot converts it to JSON / Plain Text
5. Response is sent back to the client

★ @RestController Example

```
import org.springframework.web.bind.annotation.*;

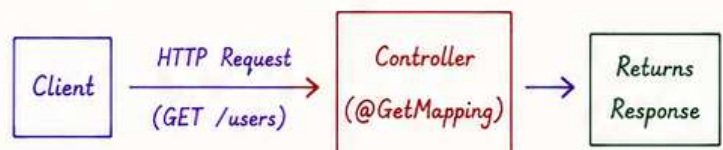
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping
    public List<String> getUsers() {
        return List.of("Aman", "Rahul", "Sonu");
    }
}
```

★ @PostMapping Example

```
@PostMapping
public String createUser(@RequestBody User user) {
    // logic to save user in database
    return "User created successfully";
}
```

★ Request Flow (Part 1)



★ Key Points

- **@RestController** = **Controller** + **ResponseBody**
- **@RequestMapping** defines the base path.
- **@GetMapping** is used to fetch data.
- **@PostMapping** is used to create data.
- Methods return data which is automatically converted to JSON.
- Spring Boot handles conversion and response.

REST API NOTES

CHAPTER 1

Page 8/10

8. BUILDING REST APIs (SPRING BOOT) - PART 2 @aman_views

★ Overview

In this part we will see other important annotations and how to handle path, query params and request body.

★ Important Annotations (Part 2)

1. @PutMapping

- Handles HTTP PUT requests.
- Used to update entire resource.

2. @DeleteMapping

- Handles HTTP DELETE requests.
- Used to delete a resource.

3. @PathVariable

- Used to capture values from URL path.
- Example: /users/10 → 10 is path variable

4. @RequestParam

- Used to capture query parameters.
- Example: /users?name=Aman&page=1

5. @RequestBody

- Used to bind request body to Java object.
- Commonly used in POST and PUT.

★ @PutMapping Example

```
@PutMapping("/users/{id}")
public User updateUser(@PathVariable int id,
                      @RequestBody User user) {
    // update logic
    return user;
}
```

★ @DeleteMapping Example

```
@DeleteMapping("/users/{id}")
public String deleteUser(@PathVariable int id) {
    // delete logic
    return "User deleted successfully";
}
```

★ @RequestParam Example

```
@GetMapping("/users")
public List<User> getUsers(@RequestParam String name,
                          @RequestParam(defaultValue = "1") int page) {
    // fetch users
    return userList;
}
```

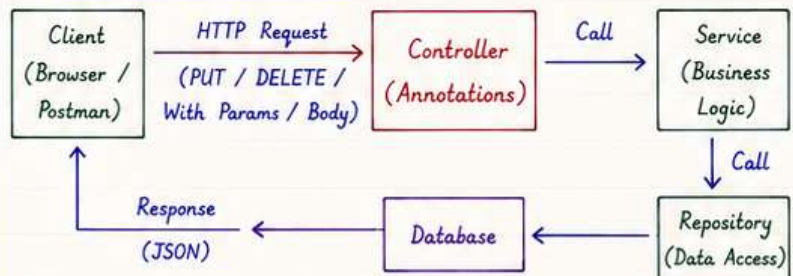
★ @RequestBody Example

```
@PostMapping("/users")
public String createUser(@RequestBody User user) {
    // save user
    return "User created successfully";
}
```

★ CRUD Annotation Summary

Operation	HTTP Method	Annotation	Purpose
Create	POST	@PostMapping	Create new resource
Read (All)	GET	@GetMapping	Get all resources
Read (By ID)	GET	@GetMapping	Get resource by ID
Update	PUT	@PutMapping	Update entire resource
Delete	DELETE	@DeleteMapping	Delete resource

★ Request Flow (Part 2)



★ Key Points

- @PathVariable → for path parameters.
- @RequestParam → for query parameters.
- @RequestBody → for request body.
- PUT is idempotent (same result for multiple requests).
- DELETE should be used carefully.
- Annotations make code clean and easy to maintain.

REST API NOTES

CHAPTER 1

Page 9/10

9. ERROR HANDLING & EXCEPTIONS

@aman_views

★ Why Error Handling is Important?

- APIs can fail due to many reasons.
- Good error messages help in debugging.
- Consistent error format improves user experience.

★ Common Reasons for Errors

- Invalid input or data
- Missing required parameters
- Resource not found
- Unauthorized access
- Server or internal issues

★ HTTP Status Codes for Errors

Code	Category	Meaning
400	Bad Request	Invalid request / bad syntax
401	Unauthorized	Authentication required
403	Forbidden	No permission to access
404	Not Found	Resource not found
409	Conflict	Resource conflict
500	Internal Server Error	Something went wrong on server
502	Bad Gateway	Invalid response from upstream server
503	Service Unavailable	Server is down or temporarily unavailable

★ Error Response Structure (Example)

```
{
  "timestamp" : "2024-05-22T10:30:00Z",
  "status" : 404,
  "error" : "Not Found",
  "message" : "User with id 10 not found",
  "path" : "/users/10"
}
```

★ Best Practices

- Always return meaningful error messages.
- Use appropriate HTTP status codes.
- Do not expose internal server details.
- Keep error response format consistent.
- Log errors on server side.

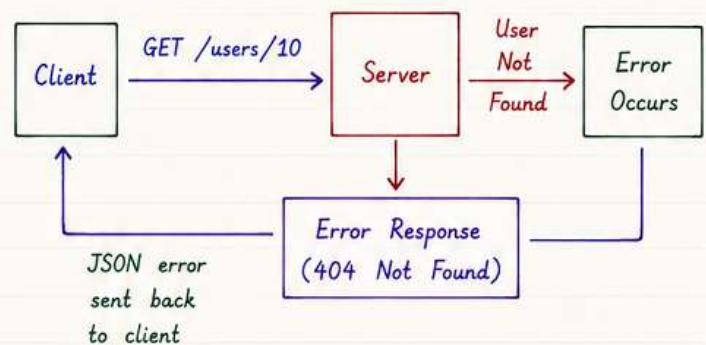
★ Global Exception Handling (Spring Boot)

@ControllerAdvice

```
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(Exception ex) {
        return new ErrorResponse(404, "Not Found",
                                ex.getMessage());
    }
}
```

★ Example Flow



★ Sample Error Response (404 Not Found)

```
{
  "timestamp" : "2024-05-22T10:30:00Z",
  "status" : 404,
  "error" : "Not Found",
  "message" : "User with id 10 not found",
  "path" : "/users/10"
}
```

★ Tips

- Validate request before processing.
- Use @Valid and custom validators.
- Return user-friendly messages.
- Monitor and alert on server errors.
- Always test error scenarios in API testing.

REST API NOTES

CHAPTER 1

Page 10/10

@aman_views

10. TESTING, DOCUMENTATION & TOOLS

★ API Testing Basics

Testing ensures that API works as expected and handles all scenarios properly.

1. **Functional Testing** - Validate endpoint functionality.
2. **Data Validation** - Validate request & response data.
3. **Negative Testing** - Invalid inputs, unauthorized access, etc.
4. **Performance Testing** - Load, stress testing.
5. **Security Testing** - Auth, input validation, data protection.

★ API Testing Checklist

- ✓ Correct status code returned
- ✓ Response time is within acceptable limit
- ✓ Response body format is correct
- ✓ All fields have correct data
- ✓ Edge cases & negative scenarios
- ✓ Authorization & authentication verified
- ✓ Pagination, filtering, sorting verified
- ✓ Error messages are meaningful

★ Sample Test Case

Test Case ID	TC_GET_USERS_01
Description	Verify GET /users returns list of users
Preconditions	User should have valid access
Request	GET /users
Expected Status	200 OK
Expected Result	List of users is returned
Test Data	N/A
Actual Result	Pass / Fail

★ Test Data Examples

Request Body (Create User)

```
{
  "name" : "Aman",
  "email" : "aman@example.com",
  "role" : "admin"
}
```

★ API Documentation

Good documentation makes API easy to understand and use.

Important Elements

- Base URL
- Endpoints & Methods
- Request Parameters (Path, Query, Header, Body)
- Response Format
- Status Codes
- Authentication Details
- Examples

★ Sample API Documentation (Example)

```
Endpoint : GET /users/{id}
Description : Get user details by ID
Method : GET
URL : /users/{id}
Path Param : id (integer) - User ID
Response : 200 OK
Response Body (JSON):
{
  "id" : 1,
  "name" : "Aman",
  "email" : "aman@example.com"
}
```

★ Popular API Testing Tools

Tool	Purpose
Postman	Manual API testing
Insomnia	API development & testing
Rest Assured	Automation testing (Java)
SoapUI	Functional & load testing
Swagger / OpenAPI	Documentation & testing

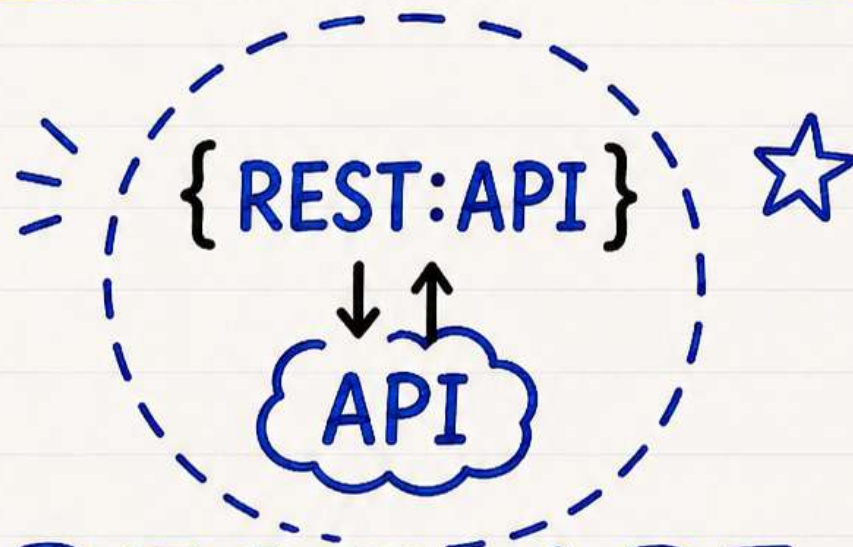
★ Key Takeaways

- Understand HTTP methods and status codes.
- Design RESTful APIs with proper conventions.
- Handle errors gracefully.
- Test APIs thoroughly.
- Document APIs clearly.
- Use the right tools to increase productivity.

REST API NOTES

— TO GET —

COMPLETE PDF



REST API

♥ SAVE | SHARE | FOLLOW ♥



COMMENT
"REST API PDF"
TO GET THE LINK



♥ > Please check the bio for < ♥
ALL NOTES LINKS IN ONE PLACE!